

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №6
по дисциплине «Организация ЭВМ и систем»
Тема: Изучение режимов адресации в ассемблере RISC-V

Студентка гр. 2304

Иванова М.А.

Преподаватель

Фирсов М. А.

Санкт-Петербург

2023

Цель работы.

Цели работы:

1. Разработка программы преобразования данных для приобретения практических навыков программирования на языке ассемблера.
2. Закрепление знаний по режимам адресации в процессоре RISC-V.

Задание. Вариант 10

1. Для заданного набора констант сформировать массив `array` из 10 элементов, в котором $arr[0] = a+b+c$, $array[i+1] = arr[i] + a + b - c$.
Доступ к массиву (инициализация, запись, чтение) должен выполняться из памяти.
2. Написать программу, которая с использованием 4 режимов адресации: регистрового, непосредственного, базового и относительного к счетчику команд реализует вычисление выражения, выбираемого из таблицы 1 в соответствии с номером студента в списке группы.

Вариант 10:

Константа	Значение
a	[Сумма цифр студ. билета 230410] = 10
b	[Количество букв в фамилии Иванова] = 7
c	[Количество букв в имени Маргарита] = 9

Номер	Вычисляемое выражение	Используемые регистры
10	ЕСЛИ ($arr[7] + arr[4] + arr[1] \leq threshold$) ТО ($res1 = arr[5] \mid arr[7]$) ИНАЧЕ ($res2 = arr[8] \mid a$)	$threshold \rightarrow a6$ $res1 \rightarrow t1$ $res2 \rightarrow s5$

Здесь `threshold` – заданный порог.

Основные теоретические положения.

1. Описание состава используемых регистров, базового набора команд и набора псевдокоманд процессора RISC-V.

Регистры:

Регистр(ы)	Наименование	Описание назначения
x0	zero	Константа 0
x1	ra	Адрес возврата
x2	sp	Указатель стека
x3	gp	Глобальный указатель
x4	tp	Указатель потока
x5-x7	t0-t2	Временные регистры
x8	s0/fp	Сохраняемый регистр / Указатель фрейма
x9	s1	Сохраняемый регистр
x10-x11	a0-a1	Аргументы функции / Возвращаемые значения
x12-x17	a2-a7	Аргументы функции
x18-x27	s2-s11	Сохраняемые регистры
x28-x31	t3-t6	Временные регистры

Использованные в ходе выполнения лабораторной работы команды процессора RISC-V:

№	Обозначение	Наименование	Тип формата	Описание
<i>Арифметические операции</i>				
1	ADD rd, rs1, rs2	Add	R	$rd \leftarrow rs1 + rs2$
2	ADDI rd, rs1, imm12	Add immediate	I	$rd \leftarrow rs1 + imm12$
3	AUIPC rd, imm20	Add upper immediate to PC	U	$rd \leftarrow PC + imm20 \ll 12$
<i>Логические операции</i>				
4	OR rd, rs1, rs2	OR	R	$rd \leftarrow rs1 \mid rs2$
<i>Операции работы с памятью</i>				
5	LW rd, imm12(rs1)	Load word	I	$rd \leftarrow mem[rs1 + imm12]$
6	SW rs2, imm12(rs1)	Store word	S	$rs2(31:0) \rightarrow mem[rs1 + imm12]$
<i>Ветвление и переходы</i>				
7	BNE rs1, rs2, imm12	Branch not equal	SB	if $rs1 \neq rs2$ $pc \leftarrow pc + imm12$
8	JAL rd, imm20	Jump and link	UJ	$rd \leftarrow pc + 4$ $pc \leftarrow pc + imm20$

Использованные в ходе выполнения лабораторной работы псевдокоманды:

№	Обозначение	Наименование	Инструкция
1	LI rd, imm12	Load immediate (near)	ADDI rd, zero, imm12
2	LA rd, sym	Load address (far)	AUIPC rd, sym[31:12] ADDI rd, rd, sym[11:0]
3	BLE rs1, rs2, offset	Branch if rs1 ≤ rs2	BGE rs2, rs1, offset
4	J offset	Unconditional jump	JAL zero, offset

2. Краткие сведения по режимам адресации в ассемблере RISC-V.

Регистровая адресация

При регистровой адресации регистры используются для всех операндов-источников и операндов-назначений (иными словами – для всех операндов и результата). Все инструкции типа R используют именно такой режим адресации.

add rd,rs1,rs2 # rd = rs1 + rs2

Непосредственная адресация

При непосредственной адресации в качестве операндов наряду с регистрами используют константы (непосредственные операнды). Этот режим адресации используют некоторые инструкции типа I, такие как сложение с 12-битной константой (addi) и логическая операция andi.

addi rd,rs1,12 # rd = rs1 + 12

andi rd,rs1,-8 # rd = rs1 & 0xFF8

Базовая адресация

Инструкции для доступа в память, такие как загрузка слова (чтение памяти) (lw) и сохранение слова (запись в память) (sw), используют базовую адресацию. Эффективный адрес операнда в памяти вычисляется путем сложения базового адреса в регистре rs1 и 12-битного смещения с расширенным знаком, являющегося непосредственным операндом. Операции загрузки (lw) – это инструкции типа I, а операции сохранения (sw) – инструкции типа S.

```
lw    rd, 36(rs1)      # rd = M[rs1+imm][0:31]
sw    rs2, 8(rs1)      # M[rs1+imm][0:31] = rs2[0:31]
```

Адресация относительно счетчика команд

Инструкции условного перехода, или ветвления, используют адресацию относительно счетчика команд для определения нового значения счетчика команд в том случае, если нужно осуществить переход. Смещение со знаком прибавляется к счетчику команд (РС) для определения нового значения РС, поэтому тот адрес, куда будет осуществлен переход, называют адресом относительно счетчика команд.

```
beq   rs1,rs2,imm      # if(rs1 == rs2) PC += imm
jal   rd,imm           # rd = PC+4; PC += imm
```

Выполнение работы.

1. Для заданного набора констант сформирован массив arrau из 10 элементов, в котором $arr[0] = a+b+c$, $array[i+1] = arr[i] + a + b - c$. Доступ к массиву (инициализация, запись, чтение) выполняется из памяти.

0x10000024	98
0x10000020	90
0x1000001c	82
0x10000018	74
0x10000014	66
0x10000010	58
0x1000000c	50
0x10000008	42
0x10000004	34
0x10000000	26

Рис.1 – сегмент данных в памяти,
содержащий значения элементов массива

2. Написана программа, которая с использованием 4 режимов адресации: регистрового, непосредственного, базового и относительного к счетчику команд реализует вычисление выражения согласно варианту.

Начальные значения: константы a, b, c и порог `theresold` задаются с помощью директивы `.equ` в сегменте данных. Результат работы программы и порог `theresold` записываются в регистры согласно варианту.

Результаты тестирования программы представлены в таблице 1.

Таблица 1 – Результаты тестирования работы программы

№	Входные данные	Выходные данные	Комментарий
1	<code>threshold = -173</code>	<code>t1 = res1 = 0</code> <code>s5 = res2 = 90</code> <code>a6 = threshold = -173</code>	Верно
2	<code>threshold = 173</code>	<code>t1 = res1 = 0</code> <code>s5 = res2 = 90</code> <code>a6 = threshold = 173</code>	Верно
3	<code>threshold = 174</code>	<code>t1 = res1 = 82</code> <code>s5 = res2 = 0</code> <code>a6 = threshold = 174</code>	Верно
4	<code>threshold = 175</code>	<code>t1 = res1 = 82</code> <code>s5 = res2 = 0</code> <code>a6 = threshold = 175</code>	Верно
5	<code>threshold = 575</code>	<code>t1 = res1 = 82</code> <code>s5 = res2 = 0</code> <code>a6 = threshold = 575</code>	Верно

Стоит отметить, что значение, сравниваемое с `theresold`, постоянно и равно `arr[7] + arr[4] + arr[1] = 82 + 58 + 34 = 174`.

Выводы.

В ходе лабораторной работы закрепились знания по режимам адресации в процессоре RISC-V. Разработана программа преобразования данных для приобретения практических навыков программирования на языке ассемблера.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: LR6.S

```
# threshold -> a6
# res1 -> t1
# res2 -> s5

.text
.global lr6

lr6:
    la      s0, array           # s0 <- адрес начала array
    addi    a0, zero, a         # a0 = a
    addi    a0, a0, b           # a0 = a0 + b = a + b
    addi    a0, a0, c           # a0 = a0 + c = a + b + c
    sw      a0, 0(s0)           # M[s0] = array[0] <- a0 = a + b + c
    addi    s0, s0, 4           # s0 = s0 + 4 = адрес array[1]
    addi    s1, zero, 9         # s1 = 9 = счетчик

loop:
    addi    a0, a0, a           # a0 + a = array[9 - s1] + a =
array[i] + a
    addi    a0, a0, b           # a0 + b = array[9 - s1] + a + b =
array[i] + a + b
    addi    a0, a0, -c          # a0 - c = array[9 - s1] + a + b - c
= array[i] + a + b - c
    sw      a0, 0(s0)           # M[s0] = array[9 - s1 + 1] =
array[i+1] <- a0 = array[i] + a + b - c
    addi    s0, s0, 4           # s0 = s0 + 4 = адрес array[9 - s1 +
2] = array[i+2]
    addi    s1, s1, -1          # s1 = s1 - 1 = уменьшение счетчика
    bne     s1, zero, loop      # если s1 != 0, переход к loop

    la      s0, array           # s0 <- адрес начала array
    lw      t0, 7*4(s0)         # t0 <- M[s0 + 7*4] = array[7]
    add     a5, zero, t0        # a5 = t0 = array[7]
    lw      t0, 4*4(s0)         # t0 <- M[s0 + 4*4] = array[4]
    add     a5, a5, t0          # a5 = a5 + t0 = array[7] + array[4]
    lw      t0, 1*4(s0)         # t0 <- M[s0 + 1*4] = array[1]
    add     a5, a5, t0          # a5 = a5 + t0 = array[7] + array[4]
+ array[1]

    lw      a6, threshold       # a6 <- threshold
    ble     a5, a6, continue    # если a5 <= a6, переход к continue
    lw      s5, 8*4(s0)         # s2 <- M[s0 + 8*4] = array[8]
    addi    t0, zero, a         # t0 = a
    or      s5, s5, t0          # s5 = s5 | t0 = array[8] | a
    j       end                 # переход к end

continue:
    lw      t1, 5*4(s0)         # если a0 <= a6
    lw      t0, 7*4(s0)         # t1 <- M[s0 + 5*4] = array[5]
    or      t1, t1, t0          # t0 <- M[s0 + 7*4] = array[7]
                                # t1 = t1 | t0 = array[5] | array[7]

end:
```

```

        li      a0, 0                # a0 = 0 = код возврата
        li      a7, 10              # a7 = 10 = служебная команда
завершения работы
        ecall                       # вызов системы для завершения работы
программы

.data
array:      .word 1,2,3,4,5,6,7,8,9,10
threshold:  .word 175
.equ a,10
.equ b,7
.equ c,9

```